

Formal equivalence checking of PyTorch programs with ESBMC

BMC for fusion/rewrite correctness — and how the encoding compares to interactive proof

Lucas Cordeiro

University of Manchester

Proof-of-concept · status update (expert version)

Optimising transformations — **operator fusion, kernel rewrites, compiler passes** — must be semantics-preserving. We decide:

$$\forall X, W \text{ with entries in } [-10, 10]. \quad P_{\text{unfused}}(X, W) \equiv P_{\text{fused}}(X, W)$$

- **Testing** under-approximates: it samples a finite input set.
- We want **exhaustive** equivalence over an input region, with **bit-precise IEEE-754** (where fusion breaks: reassociation, rounding).
- Two verdicts: **proof** (UNSAT) or a **concrete counterexample** (SAT).

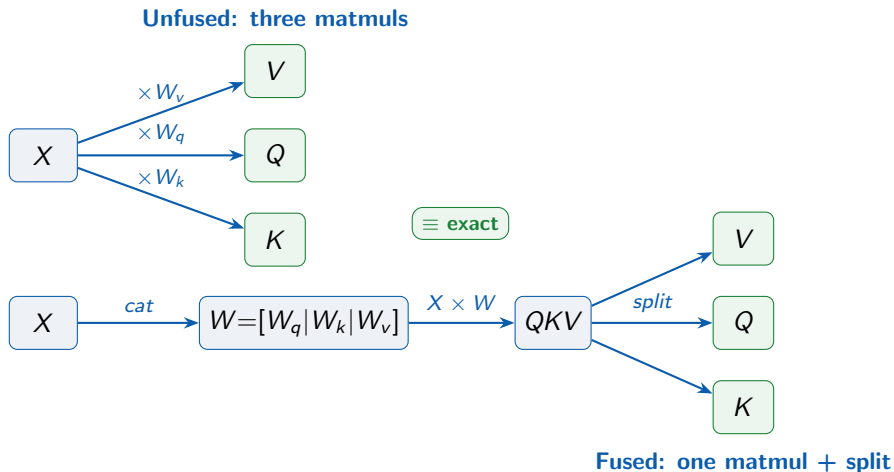
Checked under a **sequential IEEE-754 semantics**; inputs are bounded, so **NaN/Inf are excluded** — equivalence is proven over finite FP domains. **Not modelled**: backend optimisations (FMA, cuBLAS), parallel/tree reductions, scheduling.

unfused: $Q = XW_q$, $K = XW_k$, $V = XW_v$

fused: $W = [W_q \mid W_k \mid W_v]$, $QKV = XW$, split

The fused column j is the **identical multiply-add sequence** as the corresponding unfused projection $\Rightarrow$ the two are equal **bit-for-bit under the same evaluation order** (sequential IEEE-754), not merely within a tolerance. Both claims are checked.

The two programs, side by side



The program **acts as the specification** under the assumed operational model (correctness depends on the OM's faithfulness) — no separate spec to maintain.

- **Tensors** → nested lists; **ops** → operational model `torch.py` (`mm/matmul/cat/split/allclose`), pure-Python, float-typed, compiled into the ESBMC binary.
- **Inputs** → symbolic `nondet_float()` with `__ESBMC_assume` bounds (excludes NaN/Inf $\Rightarrow$ finite FP domain).
- **Shapes** → dimensions are **fixed and well-typed**; shape compatibility is assumed.
- **Property** → exact (`== / allclose(rtol=0, atol=0)`) or tolerance (`allclose` defaults).
- **Two encodings**: scalar-unrolled (no OM) *and* torch-native.

The embedding (2/2): example harness

```
X    = [[bounded() for _ in range(D)] for _ in range(S)]    # symbolic
QKV = torch.mm(X, torch.cat([Wq, Wk, Wv], dim=1))            # fused weight (cat, dim=1)
assert torch.allclose(torch.mm(X, Wq), split_Q(QKV), 0.0, 0.0)
```

Symbolic bounded inputs; the matmul goes through the operational model; the property asserts exact (zero-tolerance) equivalence of the unfused and fused Q . (*In the PoC, `cat/split` are realised by manual column indexing — `torch.cat/split` are unwinding-heavy.*)

Python frontend $\rightarrow$ **GOTO** $\rightarrow$ symbolic execution $\rightarrow$ **VCs** $\rightarrow$ SMT.

- Fixed dims $\Rightarrow$ loops fully unwound; **unwinding assertions ON** (no vacuous SUCCESS on truncation). Symbolic dims $\Rightarrow$ `--k-induction` with convergence.
- **Bit-precise IEEE-754** FP theory (Bitwuzla / Z3); rounding modelled, not abstracted.
- **UNSAT** $\Rightarrow$ **proved** over the region; **SAT** $\Rightarrow$ **falsifying assignment**.
- TCB: Python frontend + operational model + SMT solver.
- **Not modelled**: backend-specific optimisations (FMA, parallel/tree reductions, cuBLAS).
Intended use: reference-semantics equivalence, not production-kernel equivalence.

8 / 8 — QKV proved *exact* and *tolerance*, in *scalar* and *torch-native* encodings; each clean target paired with a refuted mutant (Bitwuzla, fixed dims).

Target	predicate	verdict	time
qkv_equivalence_exact	scalar ==	✓ SUCCESSFUL	6 s
qkv_equivalence	scalar tolerance	✓ SUCCESSFUL	12 s
qkv_equivalence_torch_exact	allclose(0,0)	✓ SUCCESSFUL	122 s
qkv_equivalence_torch	allclose def.	✓ SUCCESSFUL	124 s
*_buggy (×3)	refutation	✗ violated (c.ex.)	35–338 s
bias_linear (+ buggy)	$XW + b \equiv [X 1][W; b]$	✓ / ✗	35–135 s

Legend: ✓ SUCCESSFUL = property holds (UNSAT); ✗ VIOLATED = counterexample found (SAT). The *_buggy targets are intentional mutants, so a counterexample is the *desired* outcome (non-vacuity check).

Driving real PyTorch-style code through ESBMC required fixing the embedding path — **4 contributions merged upstream:**

- **torch operational model** (#5120).
- **Nested-list value/type across returns & copies** (#5111, #5113; fixes #5102/#5103).
- **Homogeneous nested-list FP element-type** (#5131; fixes #5129) — found, root-caused, and fixed by this PoC.

The *embedding cost* here was partly tool-engineering — now amortised for everyone.

ESBMC (BMC) vs Lean (ITP) + LLM agent

Aspect	ESBMC (BMC)	Lean (ITP) + LLM agent
Embedding effort	program acts as spec under the OM (write both, assert $\equiv$)	embed tensor + FP semantics, state the theorem
Automation	push-button: auto VC-gen + SMT	interactive; agent drives tactic search, proofs need guidance
FP semantics	bit-precise IEEE-754 native	modelled/axiomatised; bit-level rea- soning heavy
Counterexamples	concrete falsifying input (SAT)	none by default
Scope	bounded (fixed sizes)	unbounded / size-general
TCB	frontend + OM + solver	small proof kernel

- **Gate rewrites with BMC (ESBMC):** automatic, bit-precise, bug-finding, on concrete shapes — fast CI feedback, and a *counterexample* when it breaks.
- **Reach for ITP (Lean) for the size-general law:** $\forall$ shapes, machine-checked.
- They **compose**: BMC to find bugs and certify shipped configurations; ITP for the general theorem — where the **encoding + proof effort** and the **lack of FP counterexamples** is the real cost.

For “is this fusion correct for the shapes we deploy?”, push-button BMC with bit-precise FP and counterexamples is highly effective in practice.

- ✓ QKV fully verified (exact + tolerance, 2 encodings); bias-fused linear added.
- → **Scale**: larger/symbolic dims via k-induction; native cat/split (currently unwinding-heavy, #5121).
- → A like-for-like **Lean + agent** encoding of the same equivalence, to quantify the encoding/proof effort empirically.

Scalability is bounded by **SMT solving over floating-point arithmetic** and **loop unwinding** — cost grows rapidly with tensor size and FP-operation count.

Summary

Bounded model checking gives an **automatic, bit-precise, counterexample-producing** equivalence check for PyTorch rewrites — at the cost of being **bounded**.

The encoding cost is **relatively low once the operational model is available** (the program acts as the spec); interactive proof buys generality at a higher encoding/proof cost and **no counterexamples**.

All results are reproducible via the ESBMC harnesses and the `torch` operational model (`make verify`).